SECTION 5

3D ENVIRONMENT

ORTHOGRAPHIC PROJECTION VS. PERSPECTIVE PROJECTION

ORTHOGRAPHIC

- :Simply, it ignores the **z-component** of the point. Good for engineering designs but not realistic because if objects far from the eye should appear smaller than objects close the eye. However, this doesn't happen in orthographic projection

PERSPECTIVE

- Things far from the eye look smaller. It tries to model the viewing of **the human eye**: First, a ray from the point to the eye is drawn, and its projection on the near plane is observed. The projection on the near plane is what we see on the screen.

SET UP 3D

```
void SetTransformations()
{       //set up the logical coordinate system of the window: [-100, 100] x
[-100, 100]
        glMatrixMode(GL_PROJECTION);
     glLoadIdentity();
      gluPerspective(60,(double)800/ 600, 0.01, 1000);
      glEnable(GL_DEPTH_TEST);
// set the camera here
gluLookAt(
                  //cam pos
                  30, 40, 20,
                  //looking at
                  0, 0, 0,
                  //up dir
                  0, 1, 0);
}
```

SET UP 3D

- **fovy**
- **Specifies the field of view angle, in degrees, in the y direction.**

- **aspect**
- **Specifies the aspect ratio that determines the field of view in the x direction. The aspect ratio is the ratio of x (width) to y (height).**

- **zNear**
- **Specifies the distance from the viewer to the near clipping plane (always positive).**

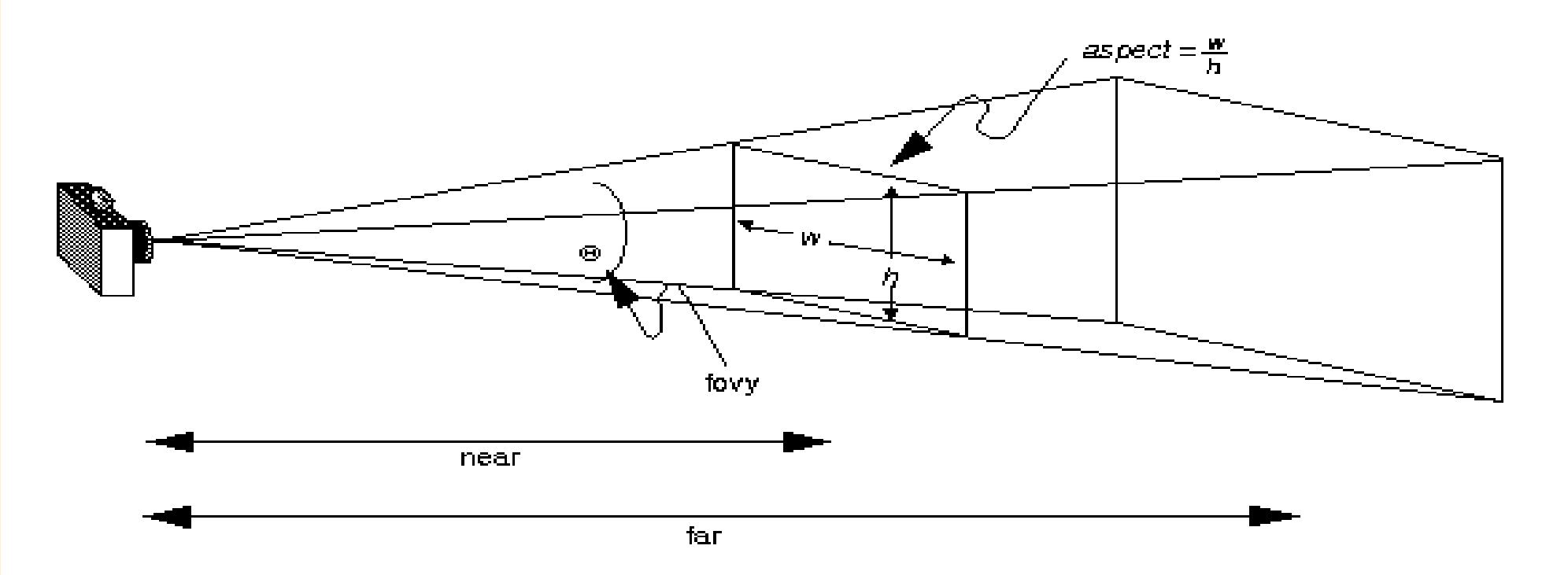
- **zFar**
- **Specifies the distance from the viewer to the far clipping plane (always positive).**

PRESPECTIVE

- The command **gluPerspective**() creates a viewing volume of the shape of a frustum, by specifying the angle of the field of view in the x-z plane and the aspect ratio of the width to height (x/y). (For a square portion of the screen, the aspect ratio is 1.0.) These two parameters are enough to determine an untruncated pyramid along the line of sight. You also specify the distance between the viewpoint and the near and far clipping planes, thereby truncating the pyramid. Note that **gluPerspective**() is limited to creating frustums that are symmetric in both the x- and y-axes along the line of sight, but this is usually what you want.

```
void gluPerspective(    GLdouble fovy,
                        GLdouble aspect,
                        GLdouble zNear,
                        GLdouble zFar);
```

PRESPECTIVE



GLENABLE()

- The command **glEnable()** enables a given OpenGL capability, in our case we are enabling the depth buffer. The depth buffer holds the depth of each pixels of the frame. It is also called z buffer.

GLULOOKAT()

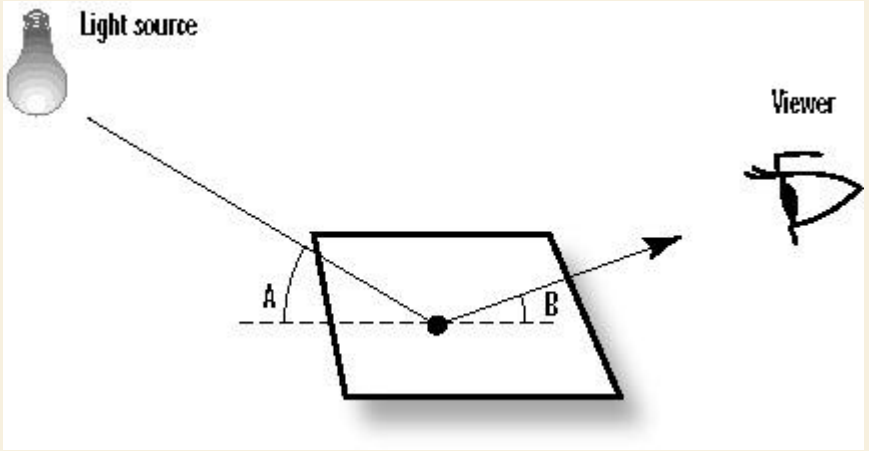
- The command **gluLookAt** creates a viewing matrix derived from an **eye** point, a reference point indicating the **center** of the scene, and an ***UP*** vector.

```
void gluLookAt(   GLdouble eyeX,
                  GLdouble eyeY,
                  GLdouble eyeZ,
                  GLdouble centerX,
                  GLdouble centerY,
                  GLdouble centerZ,
                  GLdouble upX,
                  GLdouble upY,
                  GLdouble upZ);
```

EXAMPLE

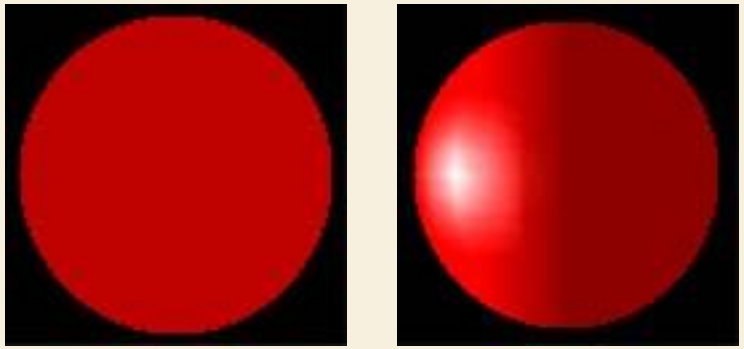
LIGHTENING

LIGHTENING



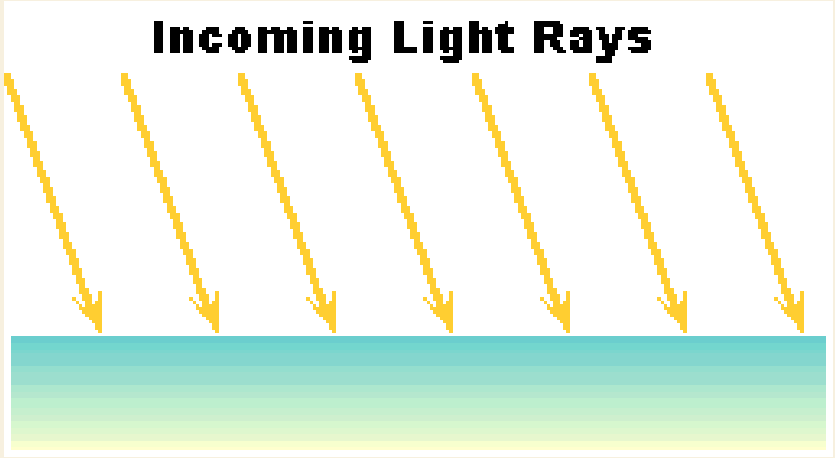
LIGHTENING

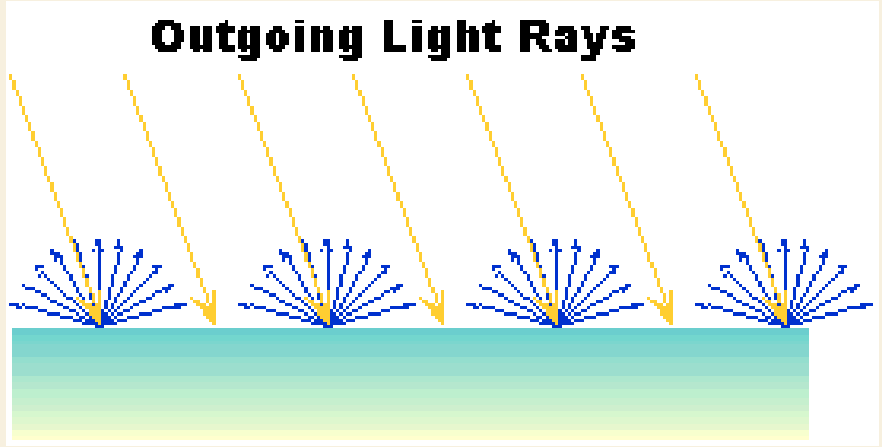
- To use **lighting** in OpenGL, you have to:
  - define one or more light sources, and
  - specify a material for each primitive drawn that describes how the primitive reflects the incident light



TYPES OF REFLECTION

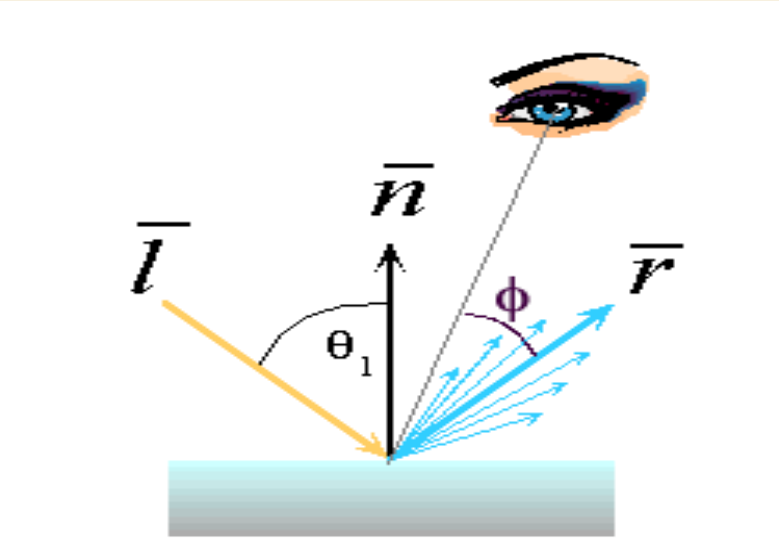
- **DIFFUSE** - An incoming ray of light is reflected uniformly in all directions. So, you see it the same from any view point. However, once the light radiates from the surface, it does so equally in all directions. It is diffuse lighting that best defines the color of 3D objects.





TYPES OF REFLECTION

- **SPECULAR** – Specular reflection is similar to reflections of light off a shiny, mirror-like surface. Specular lighting helps us to distinguish between flat, dull surfaces such as plaster and shiny surfaces like polished plastics and metals.



TYPES OF REFLECTION

- **AMBIENT** – Some of the light rays from a light source undergo multiple reflections from various objects in the surroundings and from light sources that populate the environment before hitting and reflecting off the surface of the observed object. But it would be computationally very expensive to model this kind of light precisely. OpenGL approximates this by the **ambient** light which is a uniform background light existing everywhere in the scene. Similar to diffuse, the light is reflected off the surface uniformly in all directions.

- **EMISSION** – The polygon itself emits additional light regardless of the light sources in the scene.

LIGHT SOURCES IN OPENGL

With OpenGL, you can define up to eight light sources, which are referred to through names GL_LIGHT0, GL_LIGHT1, etc. Each source is invested with various properties, and must be enabled. Each property has a default value. to create a source located at (3, 6, 5) in world coordinates, use:

```
float myLightPosition[4] = {3.0, 6.0, 5.0, 1.0};
glLightfv(GL_LIGHT0, GL_POSITION, myLightPosition);
glEnable(GL_LIGHTING); // enable
glEnable(GL_LIGHT0); // enable this particular source
```

LIGHT SOURCES IN OPENGL

- The command used to specify all properties of lights is **glLight**{if}[v](light, pname, param), where:

Light	refers to the light source being modified. It can be either GL_LIGHT0, GL_LIGHT1, …, or GL_LIGHT7.
Pname	refers to the characteristic of the light being set. It can be any of the named parameters given in table 1.
Param	indicates the values to which the pname characteristic is set. It can be an array of values if you are calling the vector version glLight{if}v (). If you the suffix v is dropped, it should be a single value.

MATERIAL PROPERTIES

- The material properties define how the surface of the primitive reflects the incident light.
- All material properties can be set using the command **glMaterial**{if}[v](face, pname, param), where

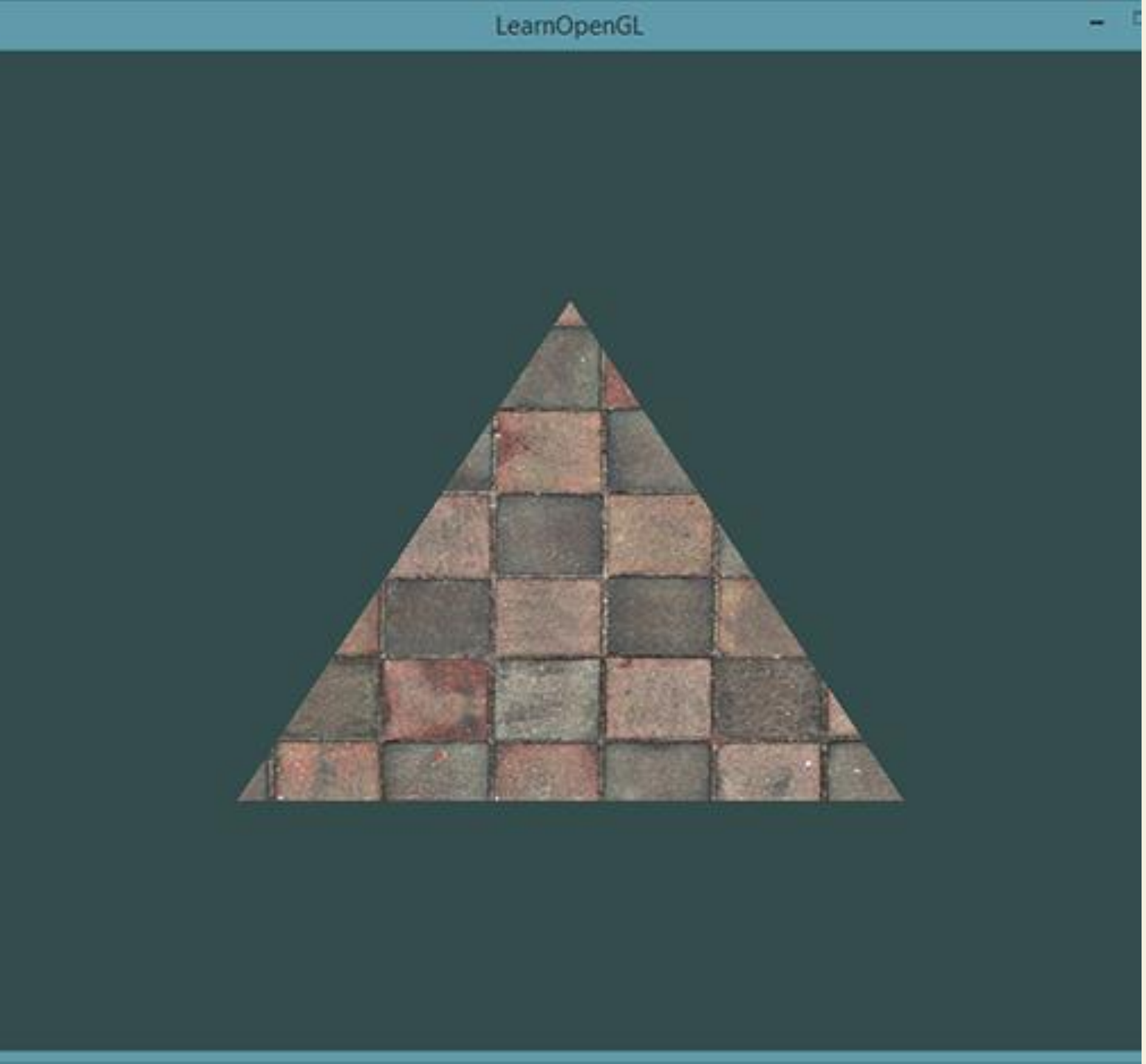
Face	can be GL_FRONT, GL_BACK, or GL_FRONT_AND_BACK to indicate which face of the object the material should be applied to. A primitive is front-facing when its vertices appear to the camera in a counter-clockwise order. Otherwise, the primitive is back-facing. OpenGL allows you to put different material to front- and back-facing primitives.
Pname	refers to the particular material property being set.
Param	is the desired values for that property, which is either an array of values (if the vector version is used) or the actual value (if the nonvector version is used). The nonvector version works only for setting GL_SHININESS. The possible values for pname are shown in table 2. Note that GL_AMBIENT_AND_DIFFUSE allows you to set both the ambient and diffuse material colors simultaneously to the same RGBA value.

EXAMPLE

TEXTURE

TEXTURE

- What artists and programmers generally prefer is to use a **texture**. A texture is a 2D image (even 1D and 3D textures exist) used to add detail to an object; think of a texture as a piece of paper with a nice brick image (for example) on it neatly folded over your 3D house so it looks like your house has a stone exterior. Because we can insert a lot of detail in a single image, we can give the illusion the object is extremely detailed without having to specify extra vertices.

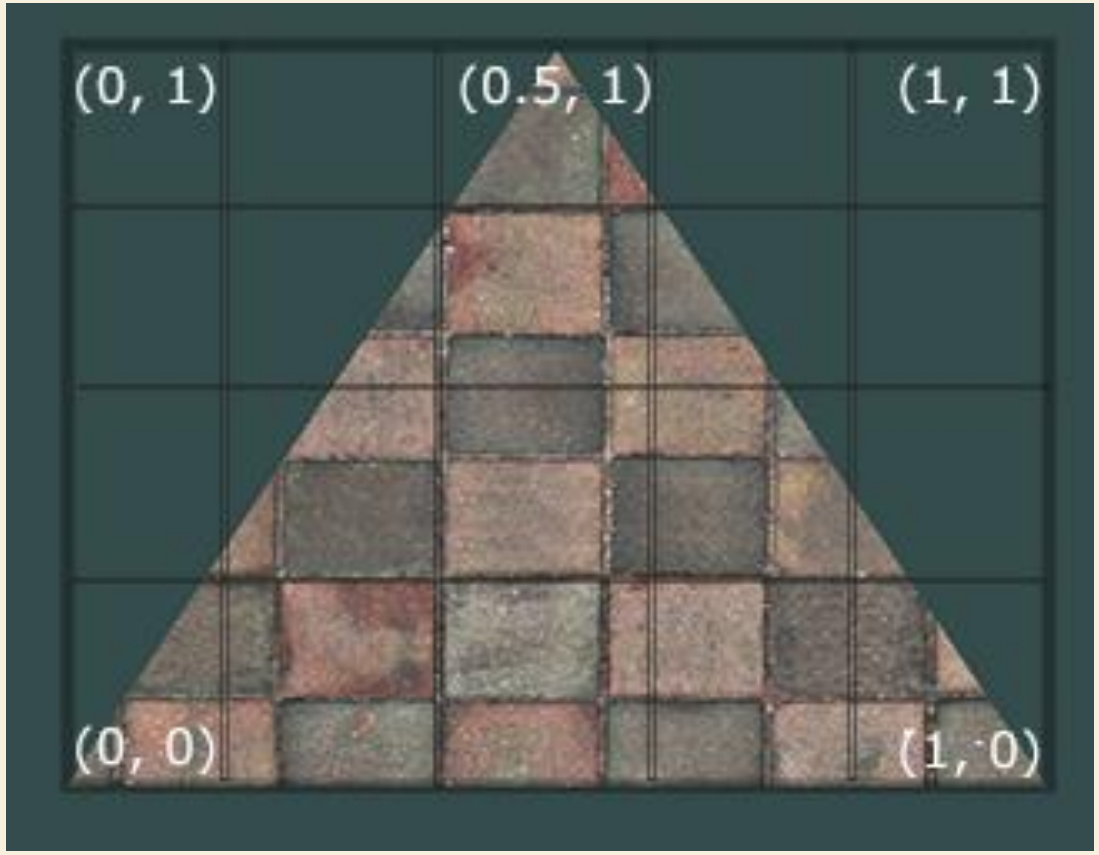


TEXTURE

In order to map a texture to the triangle we need to tell each vertex of the triangle which part of the texture it corresponds to. Each vertex should thus have a texture coordinate associated with them that specifies what part of the texture image to sample from. Fragment interpolation then does the rest for the other fragments.

TEXTURE

- Texture coordinates range from 0 to 1 in the x and y axis (remember that we use 2D texture images). Retrieving the texture color using texture coordinates is called sampling. Texture coordinates start at (0,0) for the lower left corner of a texture image to (1,1) for the upper right corner of a texture image. The following image shows how we map texture coordinates to the triangle.



TEXTURE WRAPPING

The default behavior of OpenGL is to repeat the texture images , but there are more options OpenGL offers:

- **GL_REPEAT:** The default behavior for textures. Repeats the texture image.
- **GL_MIRRORED_REPEAT**: Same as GL_REPEAT but mirrors the image with each repeat.
- **GL_CLAMP_TO_EDGE**: Clamps the coordinates between 0 and 1. The result is that higher coordinates become clamped to the edge, resulting in a stretched edge pattern.
- **GL_CLAMP_TO_BORDER**: Coordinates outside the range are now given a user-specified border color.

TEXTURE WRAPPING

GL_REPEAT

GL_MIRRORED_REPEAT

GL_CLAMP_TO_EDGE

GL_CLAMP_TO_BORDER

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT);
```

TEXTURE FILTERING

- OpenGL has options for texture filtering as well. There are several options available but for now we'll discuss the most important options: **GL_NEAREST** and **GL_LINEAR**.

- **GL_NEAREST** (also known as nearest neighbor filtering) is the default texture filtering method of OpenGL. When set to GL_NEAREST, OpenGL selects the pixel which center is closest to the texture coordinate. Below you can see 4 pixels where the cross represents the exact texture coordinate. The upper-left texel has its center closest to the texture coordinate and is therefore chosen as the sampled color:

TEXTURE FILTERING

- **GL_LINEAR** (also known as (bi)linear filtering) takes an interpolated value from the texture coordinate's neighboring texels, approximating a color between the texels. The smaller the distance from the texture coordinate to a texel's center, the more that texel's color contributes to the sampled color. Below we can see that a mixed color of the neighboring pixels is returned:

TEXTURE FILTERING

GL_NEAREST

GL_LINEAR

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

GENERATING A TEXTURE

- Like any of the previous objects in OpenGL, textures are referenced with an ID; let's create one:

```
GLuint  texture; //The id of the texture
glGenTextures(1, &texture);
```

- The **glGenTextures** function first takes as input how many textures we want to generate and stores them in a unsigned int array given as its second argument. Just like other objects we need to bind it so any subsequent texture commands will configure the currently bound texture:

```
glBindTexture(GL_TEXTURE_2D, texture);
```

GENERATING A TEXTURE

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
GL_UNSIGNED_BYTE, data); glGenerateMipmap(GL_TEXTURE_2D);
```

GENERATING A TEXTURE

This is a large function with quite a few parameters so we'll walk through them step-by-step:

- The **first argument** specifies the texture target; setting this to GL_TEXTURE_2D means this operation will generate a texture on the currently bound texture object at the same target.
- The **second argument** specifies the mipmap level for which we want to create a texture for if you want to set each mipmap level manually, but we'll leave it at the base level which is 0.
- The **third argument** tells OpenGL in what kind of format we want to store the texture. Our image has only RGB values so we'll store the texture with RGB values as well.
- The **4th and 5th argument** sets the width and height of the resulting texture. We stored those earlier when loading the image so we'll use the corresponding variables.
- The **next argument** should always be 0 (some legacy stuff).
- The **7th and 8th argument** specify the format and datatype of the source image. We loaded the image with RGB values and stored them as chars (bytes) so we'll pass in the corresponding values.
- The **last argument** is the actual image data.

EXAMPLE

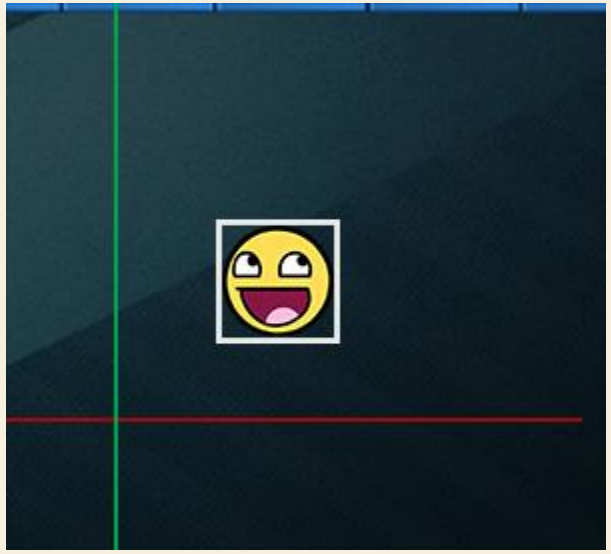
COLLISION

COLLISION

- When trying to determine if a **collision** occurs between two objects, we generally do not use the data of the objects themselves since these objects are often quite complicated; this in turn also makes the **collision** detection complicated. For this reason, it is a common practice to use more simple shapes (that usually have a nice mathematical definition) for **collision** detection that we overlay on top of the original object. We then check for collisions based on these simple shapes which makes the code easier and saves a lot of performance. Several examples of such **collision** shapes are circles, spheres, rectangles and boxes; these are a lot simpler to work with compared to meshes with hundreds of triangles.

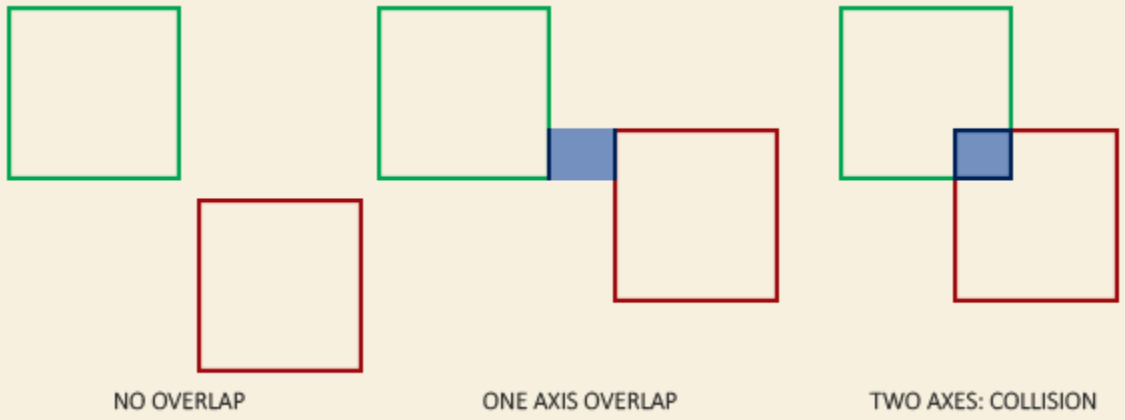
AABB - AABB COLLISIONS

- **AABB** stands for ***axis-aligned bounding*** box which is a rectangular collision shape aligned to the base axes of the scene, which in 2D aligns to the x and y axis. Being axis-aligned means the rectangular box is not rotated and its edges are parallel to the base axes of the scene (e.g. left and right edge are parallel to the y axis). The fact that these boxes are always aligned to the axes of the scene makes all calculations easier. Here we surround the ball object with an AABB:

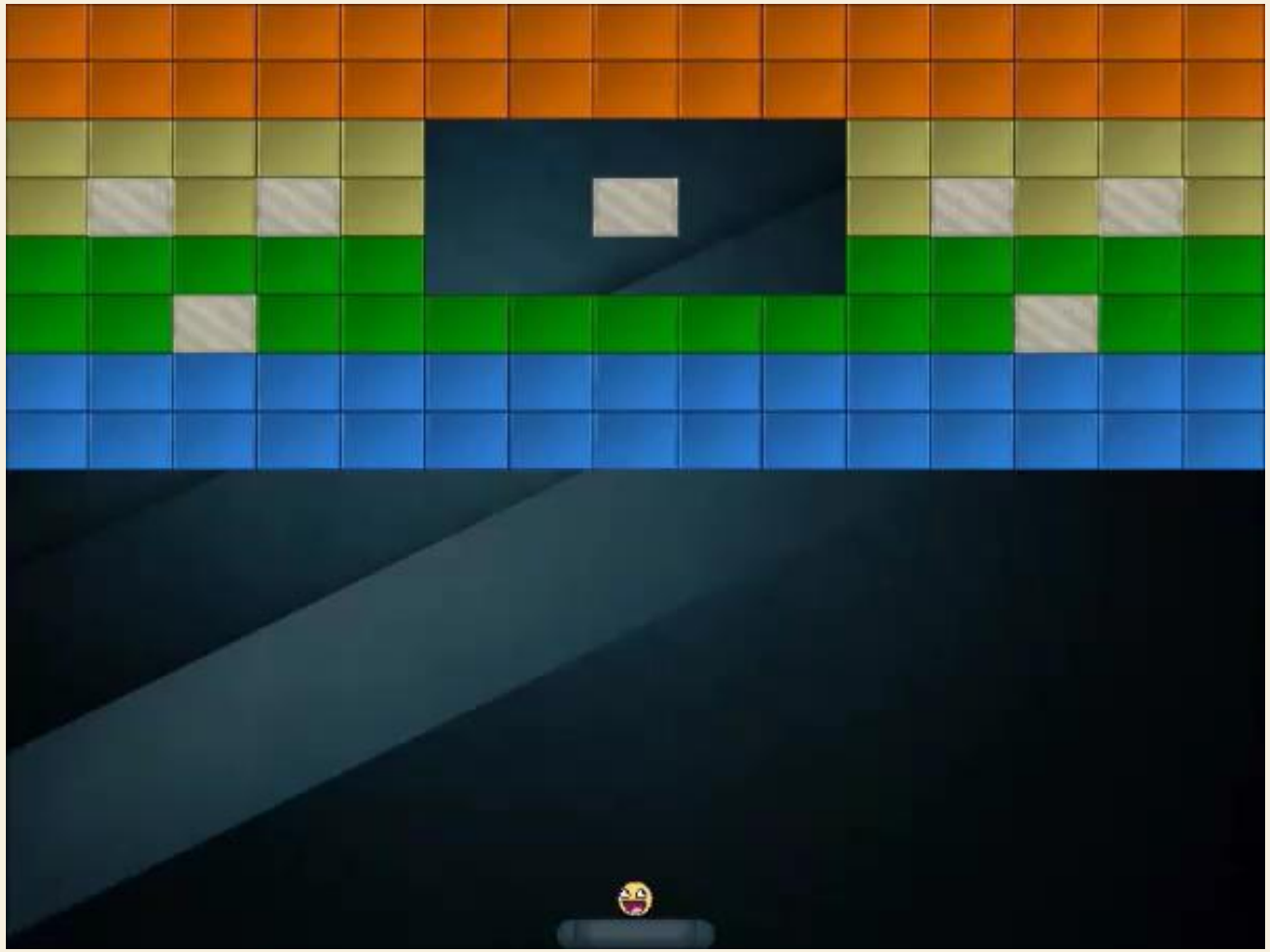


HOW DO WE DETERMINE COLLISIONS?

- A collision occurs when two collision shapes enter each other's regions e.g. the shape that determines the first object is in some way inside the shape of the second object. For **AABBs** this is quite easy to determine due to the fact that they're aligned to the scene's axes: we check for each axis if the two object' edges on that axis overlap. So basically we check if the horizontal edges overlap and if the vertical edges overlap of both objects. If both the horizontal **and** vertical edges overlap we have a collision.



HOW DO WE DETERMINE COLLISIONS?

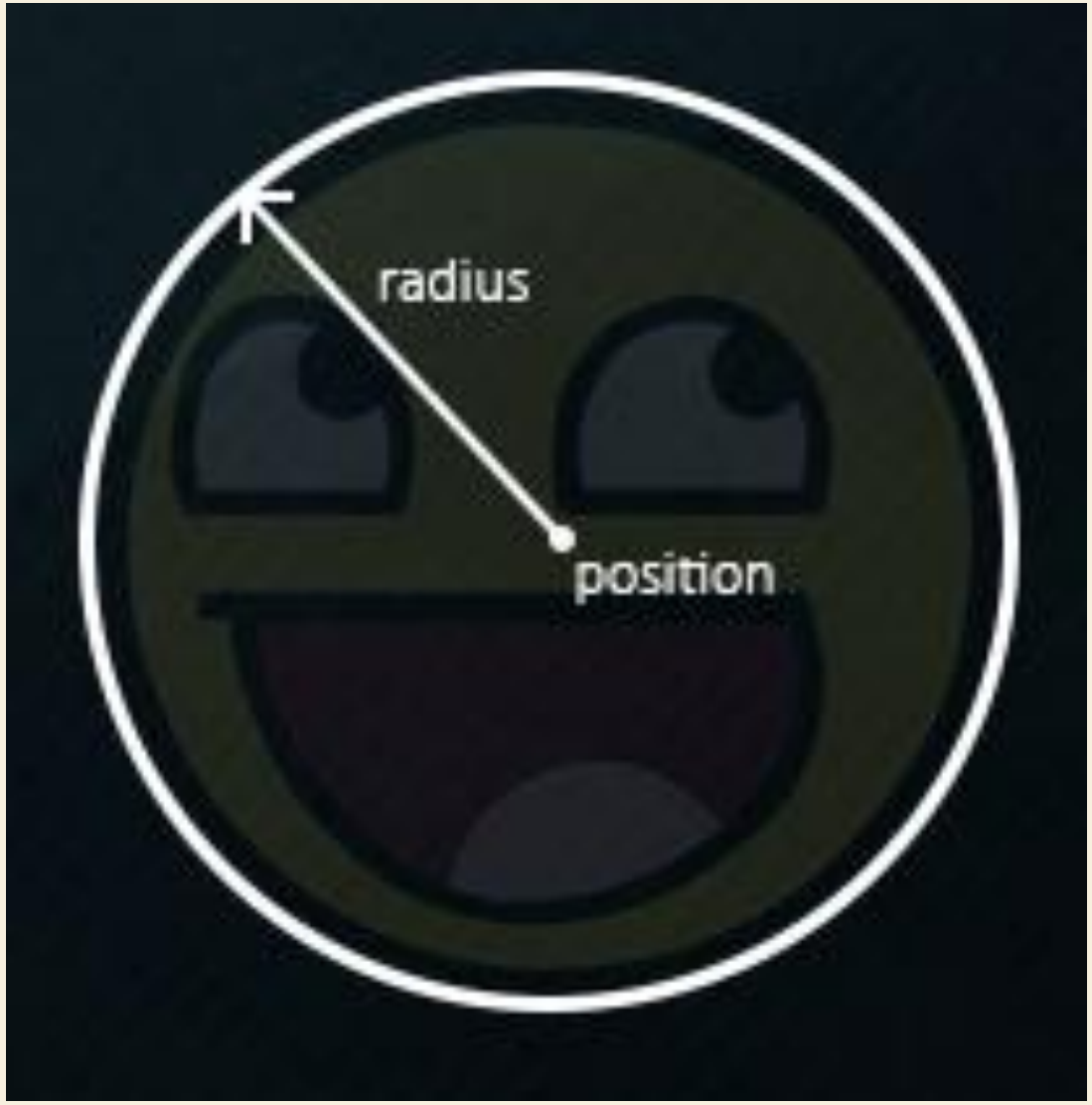


AABB - CIRCLE COLLISION DETECTION

- Beacuse the ball is a circle-like object an **AABB** is probably not the best choice as the ball's collision shape. The collision code thinks the ball is a rectangular box so the ball often collides with a brick even though the ball sprite itself isn't yet touching the brick

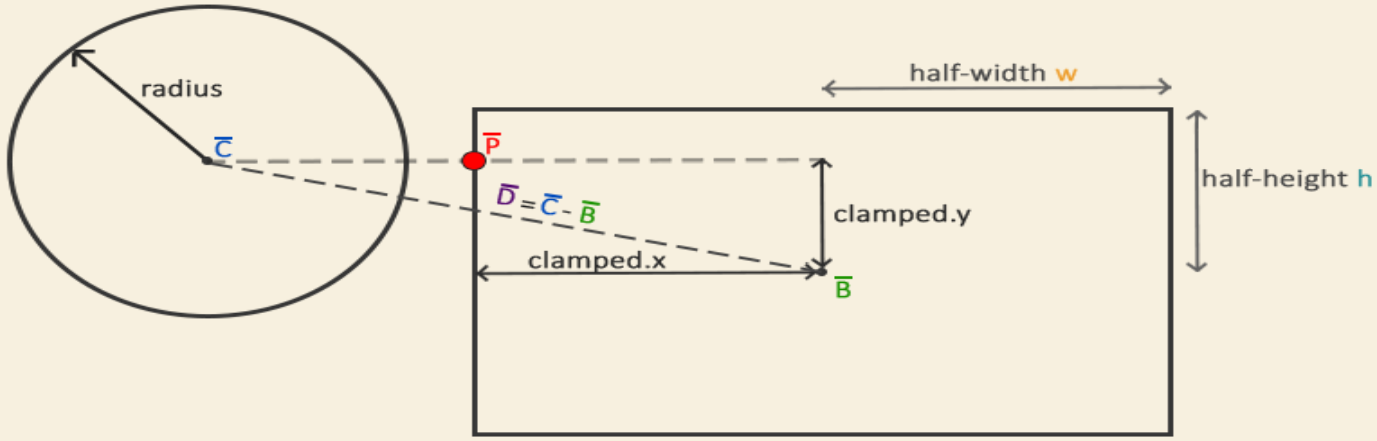
AABB - CIRCLE COLLISION DETECTION

- It makes much more sense to represent the ball with a circle collision shape instead of an **AABB**. For this reason we included a Radius variable within the ball object. To define a circle collision shape all we need is a position vector and a radius.



AABB - CIRCLE COLLISION DETECTION

- This does mean we have to update the detection algorithm since it currently only works between two **AABBs**. Detecting collisions between a circle and a rectangle is slightly more complicated, but the trick is as follows: we find the point on the **AABB** that is closest to the circle and if the distance from the circle to this point is less than its radius, we have a collision.
- The difficult part is getting this closest point $\bar{P}$ on the **AABB**. The following image shows how we can calculate this point for any arbitrary **AABB** and circle:



HOW DO WE DETERMINE COLLISIONS?

EXAMPLE

LINKS

- https://learnopengl.com/
- http://www.videotutorialsrock.com/
- https://www.ntu.edu.sg/home/ehchua/programming/opengl/CG_Introduction.html
- https://www.coders-hub.com/2013/06/opengl-code-for-collision-detection.html#.WttVyIhubIV
- https://hackr.io/tutorials/learn-opengl
- https://open.gl/
- https://www.opengl.org/resources/libraries/glut/

THANKS ☺